

Middleware in Express.js:

1. Understanding How Express.js Handles Requests

1.1 Basic Request–Response Flow

- A user (visitor) accesses a URL on the website.
- The browser sends an HTTP request to the server.
- The server runs on **Express.js**.
- Based on the requested route, Express returns:
 - An HTML response, and possibly
 - Data retrieved from MongoDB.

1.2 Introducing Middleware

Middleware is a function that executes **between the user request and the final route handler**.

Middleware enables you to:

- Apply **restrictions** or **conditional logic**.
- Allow or deny access based on:
 - Authentication,
 - Authorization,
 - Age validation,
 - Logs tracking,
 - Maintenance mode,
 - Error handling, etc.

A middleware function runs **before** the actual route handler unless configured otherwise.

2. Practical Use Cases for Middleware

2.1 Authentication Check

Example conditions:

- User must be logged in.
- User must be an administrator.

2.2 Age Validation

Example:

- Only users above 18 or 20 years can access specific routes.

2.3 Logging

Track:

- Which URL was accessed.
- When it was accessed.
- Time spent on each page.

2.4 Maintenance Mode

Redirect all routes to a temporary page.

2.5 Error Handling

Catch route-level or system-level errors and respond gracefully.

3. Creating Middleware in Express.js

3.1 Basic Syntax

A middleware function follows this signature:

```
app.use((req, res, next) => {  
  console.log("Inside Middleware");  
  next();    // Moves to the next middleware or route handler  
});
```

Parameters:

- `req` – HTTP request
- `res` – HTTP response
- `next` – function to pass control to next middleware/route

3.2 When `next()` Is Required

- Without `next()`, request will **hang** forever (loading continues).
- Middleware must call `next()` to forward the request.

4. Building Routes and Using Middleware

4.1 Example Routes

```
app.get("/", (req, res) => {
  res.send("<h1>Home Page</h1>");
});

app.get("/about", (req, res) => {
  res.send("<h1>About Page</h1>");
});
```

4.2 Applying Global (Application-Level) Middleware

```
app.use((req, res, next) => {
  console.log("Hello from Middleware");
  next();
});
```

This will run before **every** route defined below it.

5. Adding Useful Logging Inside Middleware

5.1 Log Method and URL

```
console.log(`${req.method} ${req.url}`);
```

5.2 Log Date & Time

```
const d = new Date();
console.log(`${d.getDate()}/${d.getMonth()}/${d.getHours()}:${d.getMinutes()}`);
```

6. Named Middleware for Route-Specific Use

6.1 Creating Named Middleware

```
const myMiddleware = (req, res, next) => {
  console.log("Middleware Executed");
  next();
};
```

6.2 Using on Specific Routes Only

```
app.get("/", myMiddleware, (req, res) => {  
  res.send("Home Page with Middleware");  
});
```

6.3 Using Multiple Middlewares

```
app.get("/", myMiddleware, anotherMiddleware, (req, res) => {  
  res.send("Home");  
});
```

7. Router-Level Middleware

7.1 Setting Up Router

```
const router = express.Router();
```

7.2 Creating Router-Level Middleware

```
router.use((req, res, next) => {  
  console.log("Router-Level Middleware");  
  next();  
});
```

7.3 Defining Routes Inside Router

```
router.get("/", (req, res) => res.send("Home Page"));  
router.get("/about", (req, res) => res.send("About Page"));
```

7.4 Mounting Router

```
app.use("/", router);
```

7.5 Nested Route Prefix

```
app.use("/test", router);
```

Now routes become:

- /test/
- /test/about

8. Error-Handling Middleware

8.1 Syntax (4 Parameters Required)

```
app.use((err, req, res, next) => {  
  console.log(err.stack);  
  res.status(500).send("Something broke!");  
});
```

8.2 How It Works

- Runs ONLY when an error occurs within route handlers.
- Does not run for successful requests.

8.3 Example Triggering an Error

```
app.get("/about", (req, res) => {  
  res.sed("Wrong Method"); // Typo to cause error  
});
```

9. Creating a 404 Not Found Middleware

Place at **bottom of file**, after all routes:

```
app.use((req, res) => {  
  res.status(404).send("Error 404: Page Not Found");  
});
```

10. Built-in Express Middlewares

Express provides **three** main built-in middlewares:

10.1 `express.json()`

Used for JSON body parsing (APIs).

```
app.use(express.json());
```

10.2 `express.urlencoded()`

Used when submitting HTML form data.

```
app.use(express.urlencoded({ extended: true }));
```

10.3 `express.static()`

Serve images, CSS, JS, videos.

```
app.use(express.static("public"));
```

11. Third-Party Middlewares

Express supports numerous NPM middleware packages.

Commonly Used Ones:

Package	Purpose
cors	Allow API access from other domains/devices
cookie-parser	Read client cookies
express-session	Manage sessions
multer	File upload handling
morgan	Request logging
helmet	Security hardening
passport	Authentication using providers (Google, Facebook etc.)

You can explore them at: [Resources → Middleware on expressjs.com](#).

12. Summary of Middleware Types

Type	Description
Application-Level	Runs for all routes below <code>app.use()</code>
Router-Level	Runs only for routes inside a specific router
Error-Handling	Catches and processes application errors
Built-In	Provided by Express: json, urlencoded, static
Third-Party	Installed via npm for advanced functionalities
